
PRYECTO 2 GUATERIEGOS 2.0

201318644 – Mynor Alejandro Cifuentes Velásquez

Resumen

GuateRiegos 2.0 es una plataforma web integral que permite la simulación y análisis del proceso automatizado de riego y fertilización en invernaderos mediante el uso de drones programables. El sistema, desarrollado en Python con el framework Flask, ofrece una solución robusta y moderna para la gestión eficiente de recursos agrícolas, combinando interfaz intuitiva, reportes detallados y visualización avanzada de datos y estructuras. Mediante la carga de configuraciones XML y la ejecución de planes de riego, los usuarios pueden tomar decisiones informadas y optimizar el rendimiento de sus cultivos.

Palabras clave

Riego automatizado, drones, invernadero, simulación, fertilización, eficiencia, Flask, Python, reportes, agricultura de precisión.

Abstract

GuateRiegos 2.0 is a comprehensive web platform designed for the simulation and analysis of automated irrigation and fertilization processes in greenhouses, utilizing programmable drones. Built in Python with Flask, it offers a robust and modern solution for efficient agricultural resource management, featuring an intuitive interface, detailed reports, and advanced visualization of data structures. By uploading XML configurations and executing irrigation plans, users can make informed decisions and optimize crop performance.

Keywords

Automated irrigation, drones, greenhouse, simulation, fertilization, efficiency, Flask, Python, reporting, precision agriculture.

Introducción

La agricultura actual demanda soluciones tecnológicas que permitan maximizar la productividad y optimizar el uso de recursos como agua y fertilizantes. Frente a este reto, GuateRiegos 2.0 se presenta como una aplicación innovadora que integra automatización mediante drones y una plataforma web moderna, facilitando el diseño, simulación y análisis de procesos de riego y fertilización en invernaderos. Con una estructura modular y herramientas visuales avanzadas, el sistema promueve la toma de decisiones basada en datos y la adopción de prácticas de agricultura de precisión.

Desarrollo del tema

Arquitectura y Diseño

GuateRiegos 2.0 está construido sobre una arquitectura modular que facilita su mantenimiento y escalabilidad. El backend en Python emplea el microframework Flask para la gestión de rutas, renderizado de plantillas y control de lógica de negocio. La estructura de carpetas distingue claramente los componentes: archivos de lógica principal, modelos de datos, estructuras personalizadas y vistas HTML.

Diagrama de Clases

El diagrama de clases muestra la relación entre los principales objetos del sistema: `Gestor`, `Invernadero`, `Planta`, `Dron`, `PlanRiego`, y las listas personalizadas que almacenan los diferentes elementos. Esta estructura favorece la reutilización y la extensión del sistema

permitiendo incorporar nuevas funcionalidades sin alterar el núcleo de la aplicación.

Flujo de Trabajo del Usuario

Al iniciar la aplicación, el usuario visualiza la pantalla principal donde puede cargar un archivo XML con la configuración de invernaderos, drones y plantas. La interfaz, desarrollada con HTML y Bootstrap, garantiza facilidad de uso y una experiencia amigable.

Pantalla principal para carga de XML:

Tras la carga exitosa del XML, se despliega la lista de invernaderos disponibles. El usuario selecciona uno de ellos y procede a escoger un plan de riego entre las opciones configuradas. Cada invernadero muestra información relevante como número de hileras, cantidad de plantas por hilera y los planes de riego disponibles.

Vista de selección de invernadero:

Proceso de Simulación

La simulación inicia al seleccionar un plan. Internamente, el sistema genera instrucciones para los drones asignados, considerando el orden del plan, el recorrido dentro de las hileras y la aplicación de agua y fertilizante.

```
class SimuladorRiego:
    def simular(self):
        # Crea lista de pares hilera-dron
        # Agrega instrucciones y estadísticas
        for paso in plan.pasos:
            dron = buscar_dron_para_hilera(paso.hilera)
            estado_dron = obtener_estado_dron(dron.id)

        estado_dron.instrucciones.agregar_al_final(InstruccionDron(...))
        self.estadisticas = estadisticas_dron
        self.tiempo_total = tiempo_max
```

La simulación respeta las restricciones de tiempo y recursos, asegurando que solo un dron ejecuta acciones sobre una planta en un instante dado. El sistema calcula el tiempo óptimo, el consumo total de agua y fertilizante, y genera estadísticas detalladas para cada dron.

Resultados de la simulación:

Visualización y Reportes

Una vez completada la simulación, el usuario puede acceder a reportes HTML que muestran tablas de eficiencia de drones y las instrucciones ejecutadas por tiempo. También es posible generar un archivo XML de salida con los resultados agregados para todos los invernaderos y planes.

```
<h2 class="mb-3" style="color:#800000;">Reporte de
Simulación</h2>
<h4>Invernadero: {{ invernadero.nombre }}</h4>
<table class="table table-bordered">
  <thead>
    <tr>
      <th>ID Dron</th>
      <th>Litros de Agua</th>
      <th>Gramos de Fertilizante</th>
    </tr>
  </thead>
  <tbody>
    {% for estadistica in estadisticas %}
      <tr>
        <td>{{ estadistica.dron_id }}</td>
        <td>{{ estadistica.litros_agua }}</td>
        <td>{{ estadistica.gramos_fertilizante }}</td>
      </tr>
    {% endfor %}
  </tbody>
</table>
```

La interfaz también permite al usuario definir un tiempo “t” para analizar el estado de los drones y las instrucciones ejecutadas hasta ese instante. Esta característica es clave para evaluar la eficiencia del algoritmo y entender el comportamiento dinámico del sistema.

Graficación de Estructuras de Datos

GuateRiegos 2.0 integra la herramienta Graphviz para la visualización del estado de las estructuras de datos (TDAs) durante la simulación. El usuario puede generar grafos que muestran la relación entre drones, instrucciones y plantas en cualquier momento del proceso.

Diagrama de Actividad UML – Simulación de Riego y Fertilización:

Diagrama de Actividad UML – Asignación de Drones a Hileras:

Graficación de TDAs

```
from graphviz import Digraph

def graficar_tda_simulador(simulador, tiempo,
ruta_salida='static/grafico_tda.png'):
    dot = Digraph(comment='Estado de los TDAs en tiempo t',
format='png')
    dot.attr(rankdir='LR')
    # Añade nodos y aristas por dron/instrucción
    dot.render(ruta_salida, format='png', cleanup=True)
```

Estas visualizaciones ayudan en la depuración, documentación y comprensión del funcionamiento interno del sistema

Estructuras de Datos Personalizadas

El proyecto emplea listas enlazadas simples y dobles circulares para modelar colecciones de objetos como plantas, drones y planes de riego, evitando el uso de colecciones estándar de Python y promoviendo el aprendizaje y control sobre la estructura.

```
class ListaSimple:
    def agregar_al_final(self, dato):
        nuevo_nodo = Nodo(dato)
        if not self.primerono:
            self.primerono = nuevo_nodo
        else:
            actual = self.primerono
            while actual.siguiente:
                actual = actual.siguiente
            actual.siguiente = nuevo_nodo
```

Nodo

```
class Nodo:
    def __init__(self, dato):
        self.dato = dato
        self.siguiente = None
        self.anterior = None
```

Estas estructuras permiten recorrer y modificar los elementos de manera eficiente, facilitando la implementación de algoritmos personalizados para la simulación.

Instalación y Ejecución

GuateRiegos 2.0 puede instalarse y ejecutarse en diferentes sistemas operativos gracias al uso de entornos virtuales y dependencias bien definidas en el archivo requirements.txt.

Instalación en Windows

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
```

Instalación en Ubuntu/Linux

```
python App.py # o python3 App.py en Linux
```

Extensibilidad y Mantenimiento

El diseño modular y el uso de templates HTML facilita la extensión y mantenimiento. Para agregar nuevos reportes, basta con crear funciones similares a las de generación HTML/XML existentes. Los diagramas y grafos pueden actualizarse fácilmente agregando nodos/aristas en los scripts correspondientes.

La documentación detallada, el uso de buenas prácticas de programación y la separación clara de componentes aseguran una curva de aprendizaje baja y una adopción efectiva en entornos reales.

Experiencia del Usuario y Buenas Prácticas

La interfaz guía al usuario en cada paso del proceso, desde la carga del XML hasta el análisis de resultados. Mensajes claros, validaciones y enlaces a documentación y soporte aseguran que el usuario pueda resolver problemas y aprovechar al máximo la herramienta.

Conclusiones

GuateRiegos 2.0 constituye un avance sustancial en la gestión agrícola, integrando automatización, simulación y análisis en una sola plataforma. El desarrollo en Python y Flask, la utilización de estructuras de datos personalizadas y la generación de reportes y visualizaciones avanzadas posicionan al

sistema como una herramienta estratégica para agricultores y técnicos. La modularidad, extensibilidad y documentación aseguran su adaptabilidad a futuros desafíos, contribuyendo a la eficiencia, sostenibilidad y modernización del sector agropecuario.

Extensión: de cuatro a siete páginas como máximo

Adicionalmente, se pueden agregar apéndices con modelos, tablas, etc. Que complementan el contenido del trabajo.

Referencias bibliográficas

1. Documentación oficial de Flask:
<https://flask.palletsprojects.com/>
2. Graphviz:
[https://graphviz.gitlab.io/ProyectoGuateRiegos 2.0](https://graphviz.gitlab.io/ProyectoGuateRiegos2.0), Mynor Cifuentes,
3. https://github.com/MynorCifuentes/IPC2_Proyecto2_201318644
4. Manual Técnico y Manual de Usuario de GuateRiegos 2.0.